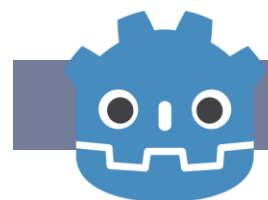


게임엔진 기초

Godot Animation Blending

강영민

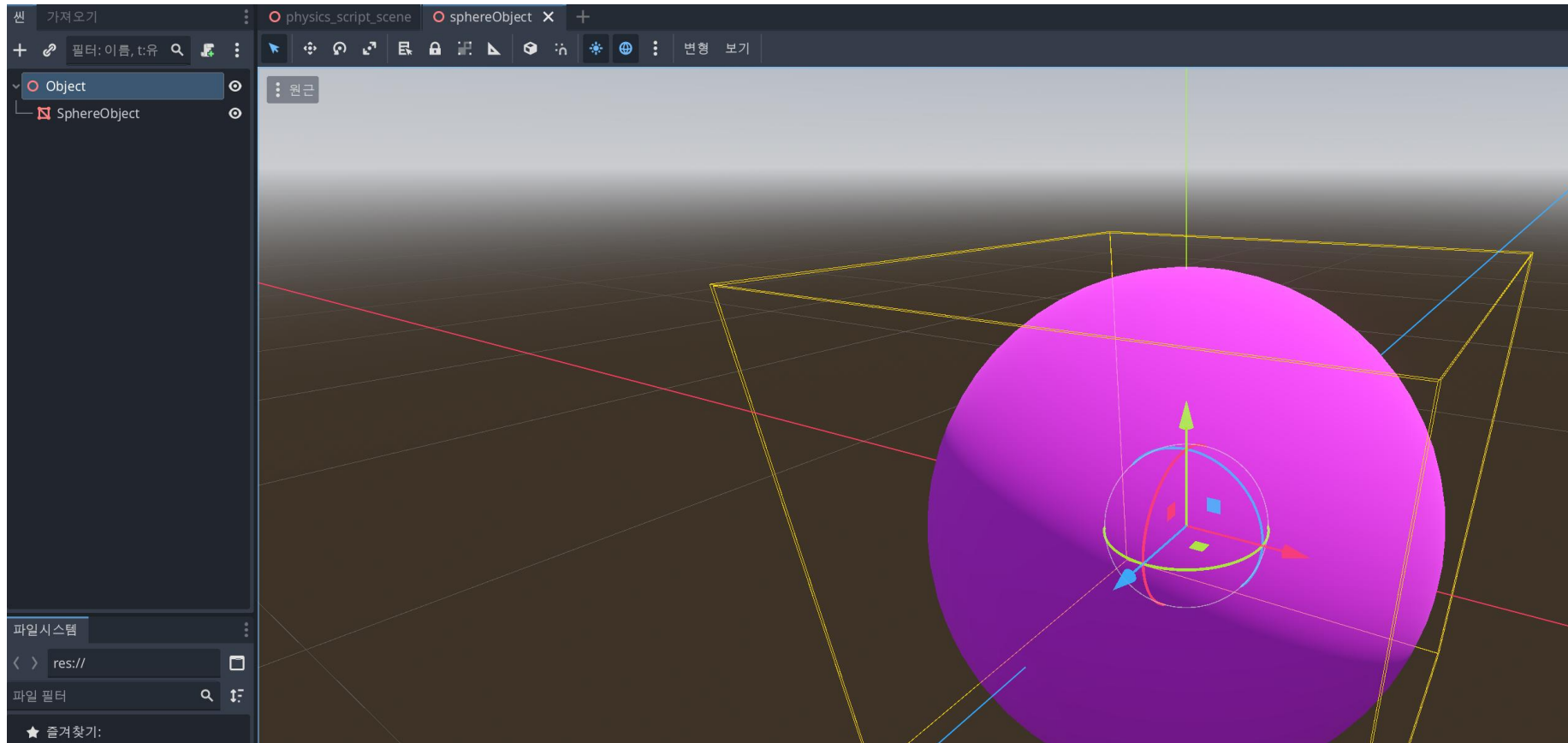
동명대학교 게임그래픽학과



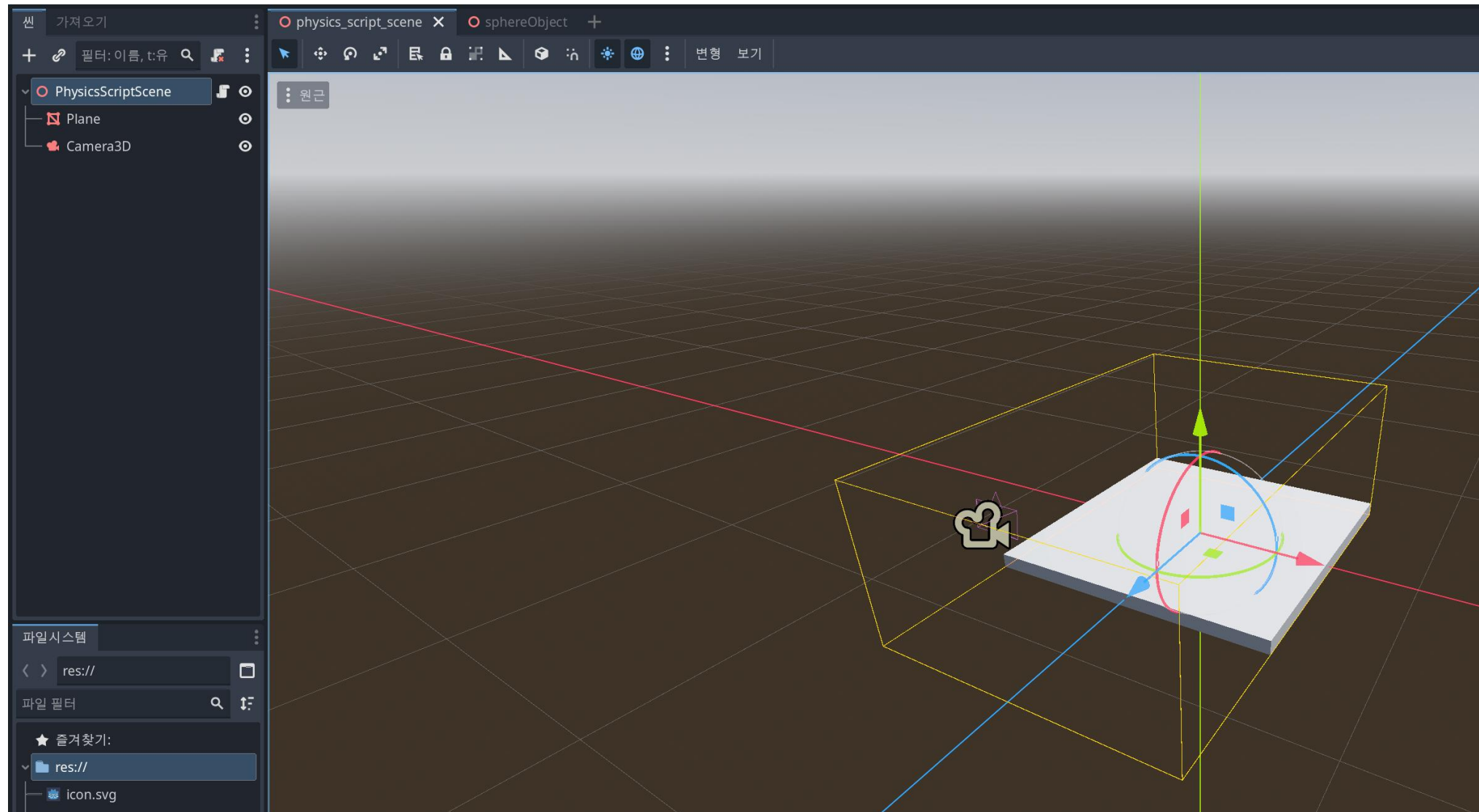
GODOT

엔진을 활용한 물리 애니메이션의 이해

공을 하나 만들어 tscn으로 저장하자: sphereObject.tscn



이 공을 사용할 장면을 만든다



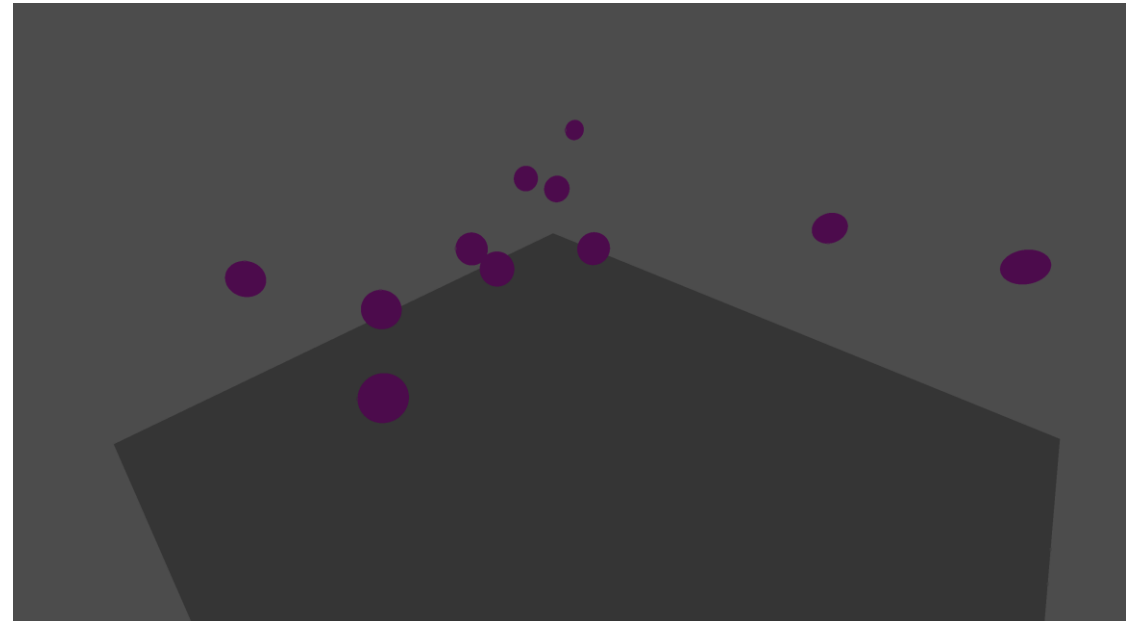
이 장면에 스크립트를 부착: 마우스 클릭 ➔ 공 생성 (랜덤 위치)

```
extends Node3D
```

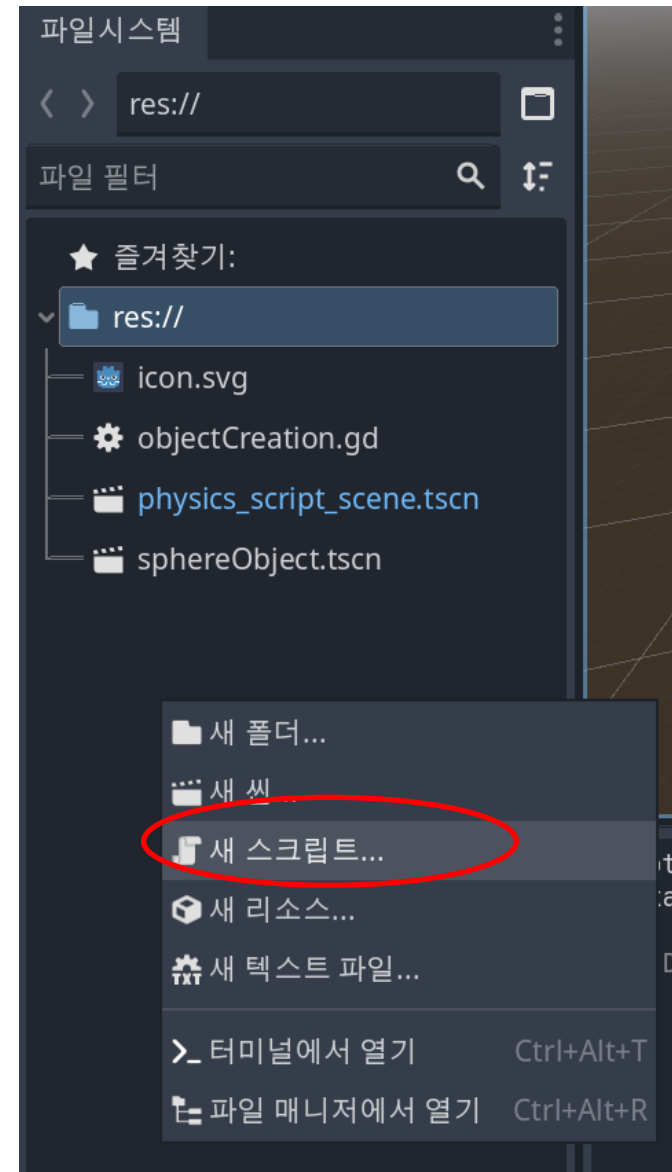
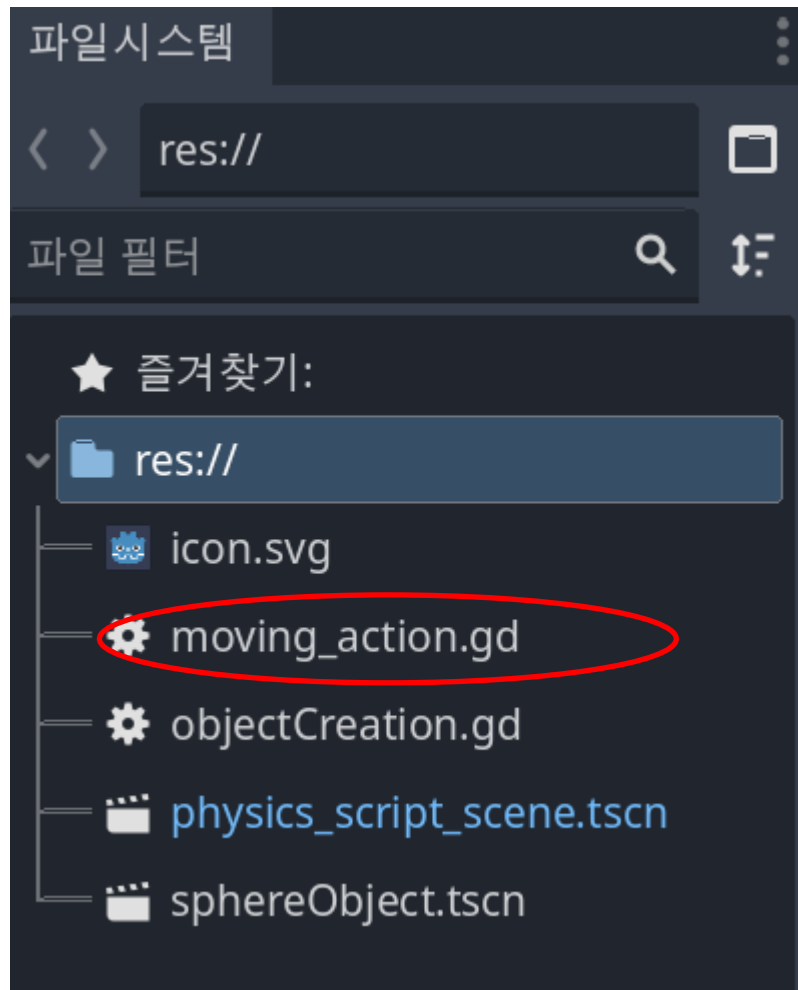
```
@export var sphere_scene: PackedScene = preload("res://sphereObject.tscn")
```

```
func _input(event):  
    if event is InputEventMouseButton \  
        and event.button_index == MOUSE_BUTTON_LEFT \  
        and event.pressed:  
        _spawn_sphere()
```

```
func _spawn_sphere():  
    var sphere = sphere_scene.instantiate()  
    sphere.global_position = Vector3(  
        randf_range(-10.0, 10.0),  
        10.0,  
        randf_range(-10.0, 10.0)  
    )  
    get_tree().current_scene.add_child(sphere)
```



물리 동작을 하는 스크립트 생성



물리 동작 구현

```
extends Node3D
```

```
var velocity := Vector3.ZERO
```

```
const GRAVITY = -20.0 # 위 코드와 동일하게
```

```
func _process(delta):
```

```
    # 중력 적용 (가속도)
```

```
    velocity.y += GRAVITY * delta
```

$acc = d\ vel / dt \rightarrow d\ vel = acc * dt$

```
    # 속도에 따라 위치 이동
```

```
    global_position += velocity * delta
```

$vel = d\ pos / dt \rightarrow d\ pos = vel * dt$

```
    # 바닥에 닿으면 멈추거나 튕기고 싶으면 여기서 처리
```

```
    if global_position.y < 0:
```

```
        global_position.y = 0
```

```
        velocity.y = -velocity.y * 0.7 # 탄성 70%
```

```
        velocity.x *= 0.8 # 마찰
```

```
        velocity.z *= 0.8
```

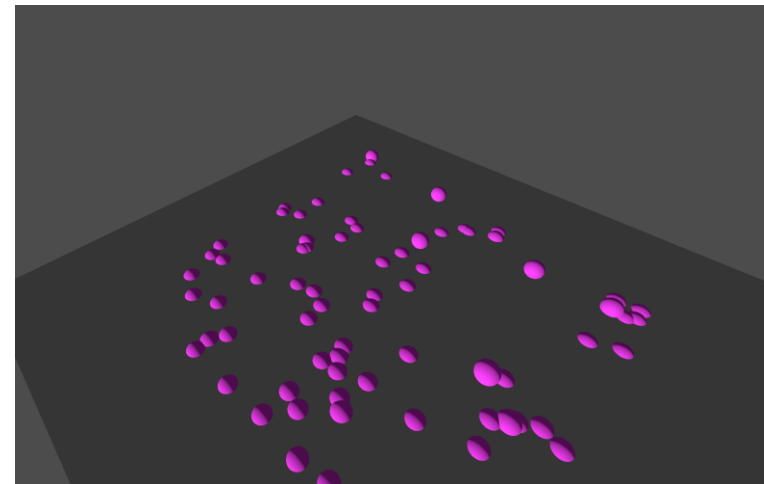
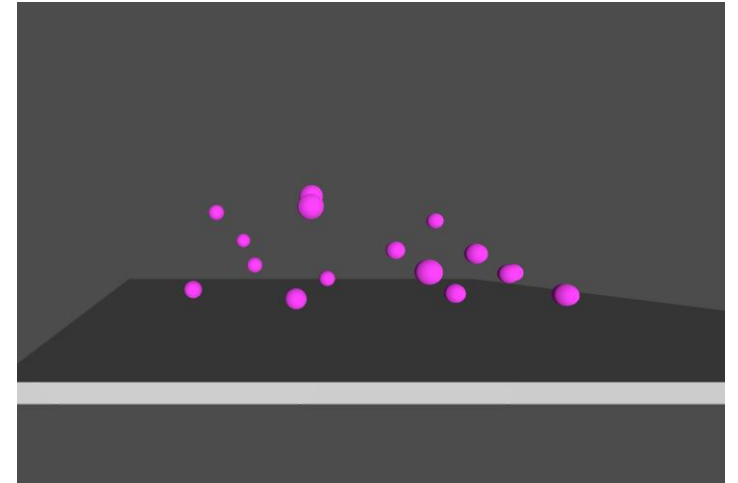
```
    # 너무 느려지면 완전히 멈춤
```

```
    if velocity.length() < 0.5:
```

```
        set_process(false) # 더 이상 움직이지 않음
```

물체 생성 코드 수정: 물리동작을 생성된 객체에 부여

```
func _spawn_sphere():  
    var sphere = sphere_scene.instantiate()  
    sphere.global_position = Vector3(  
        randf_range(-10.0, 10.0),  
        10.0,  
        randf_range(-10.0, 10.0)  
    )  
  
    # 이 스크립트를 sphere 자체에 붙여서 _process에서 움직이게 함  
    sphere.set_script(preload("res://moving_action.gd")) # 아래 코드 저장할 파일명  
  
    get_tree().current_scene.add_child(sphere)
```



공들이 서로 부담하게 하려면 → 흠...



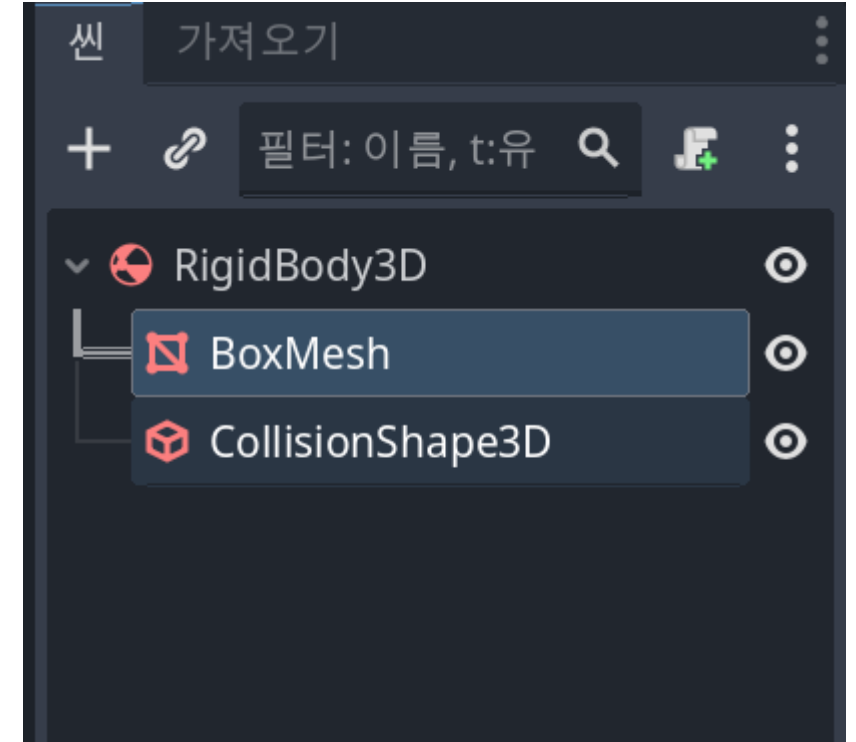
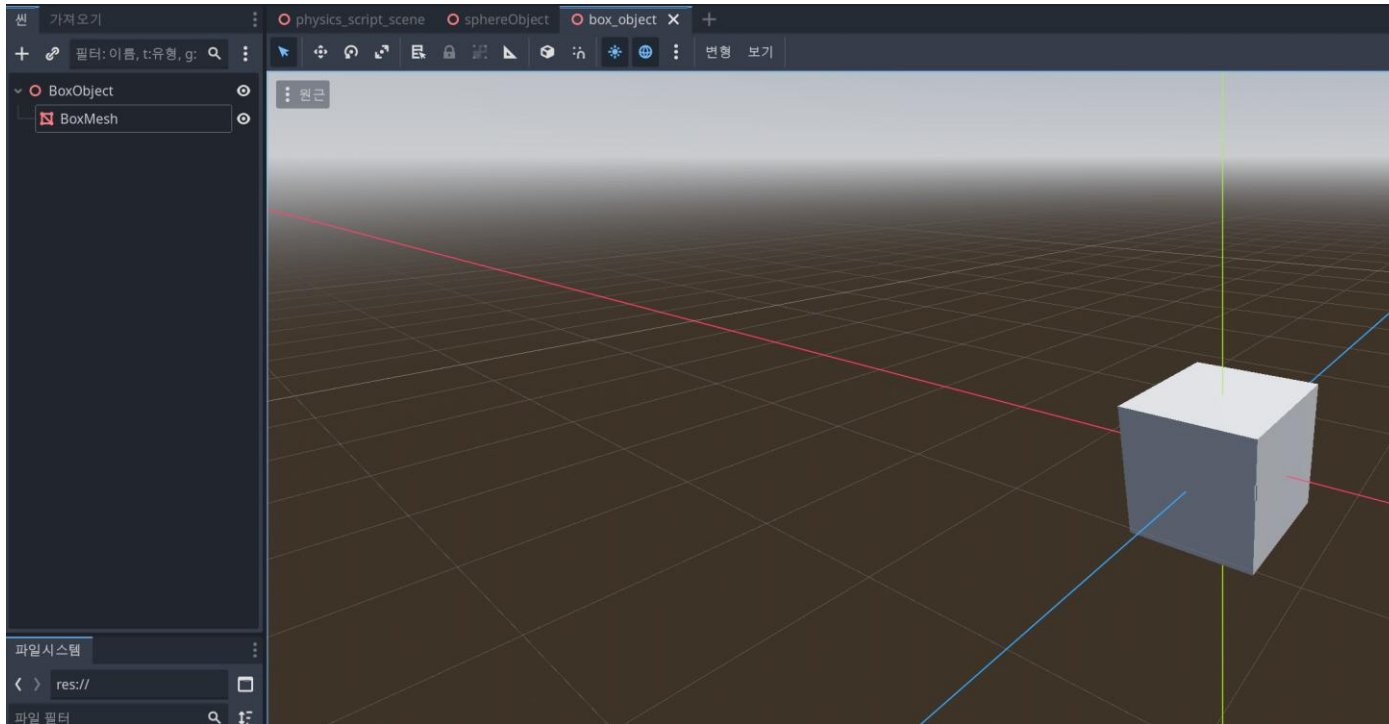
공이 아니라 박스라면... 이리 저리 튀어야 할텐데.



RigidBody2D / RigidBody3D!



박스를 만들어 보자: box_object.tscn 으로 저장



루트 노드가 RigidBody3D가 되도록 생성

메인 코드 변경

```
@export var sphere_scene: PackedScene = preload("res://sphereObject.tscn")
@export var box_scene: PackedScene = preload("res://box_object.tscn")

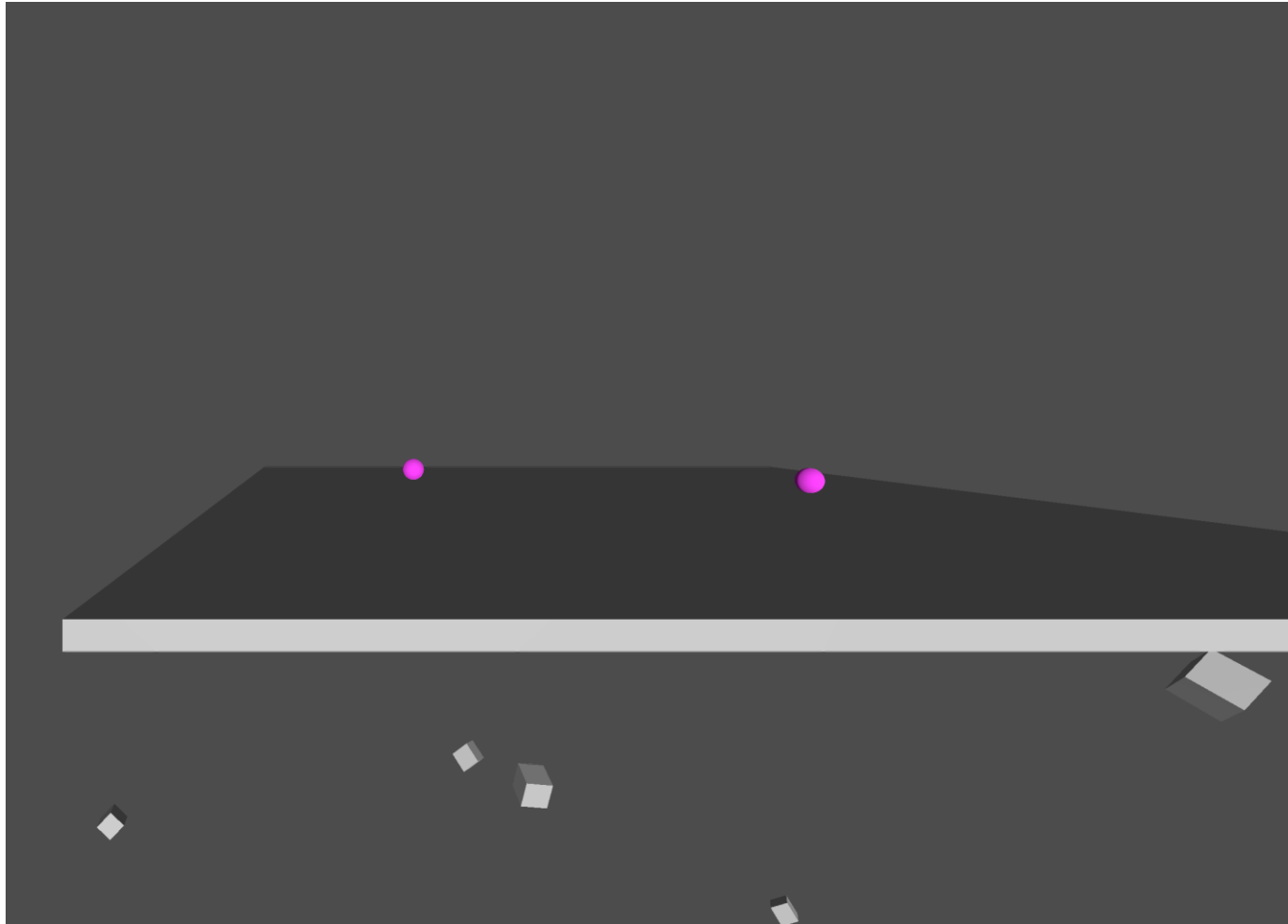
func _input(event):
    if event is InputEventMouseButton \
        and event.button_index == MOUSE_BUTTON_LEFT \
        and event.pressed:
        _spawn_sphere()

    if event is InputEventMouseButton \
        and event.button_index == MOUSE_BUTTON_RIGHT \
        and event.pressed:
        _spawn_box()
```

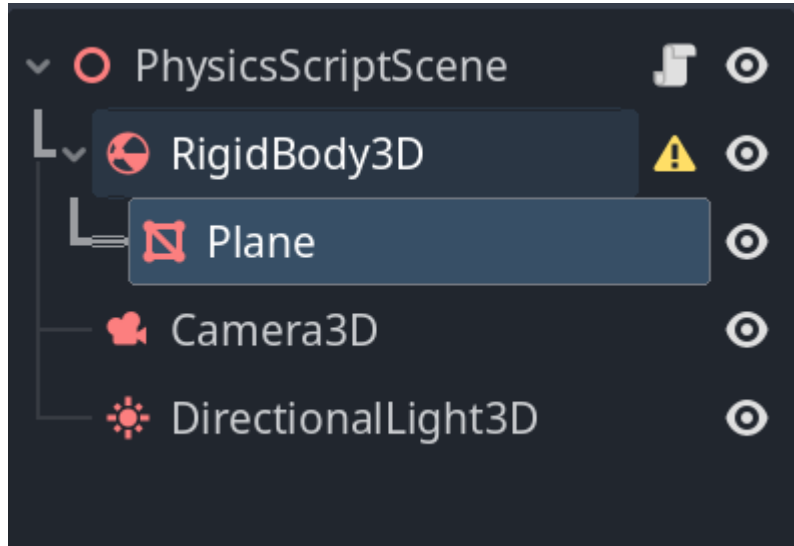
메인 코드 변경

```
func _spawn_box():  
    var obj = box_scene.instantiate() as RigidBody3D  
  
    obj.global_position = Vector3(  
        randf_range(-10, 10),  
        10,  
        randf_range(-10, 10)  
    )  
  
    obj.linear_velocity = Vector3(randf_range(-8,8), randf_range(-5,0), randf_range(-8,8))  
    obj.angular_velocity = Vector3(randf_range(-15,15), randf_range(-15,15), randf_range(-15,15))  
  
    # 색깔도 랜덤  
    if obj.has_node("MeshInstance3D"):  
        var mat = StandardMaterial3D.new()  
        mat.albedo_color = Color(randf(), randf(), randf())  
        obj.get_node("MeshInstance3D").set_surface_override_material(0, mat)  
  
    get_tree().current_scene.add_child(obj)
```

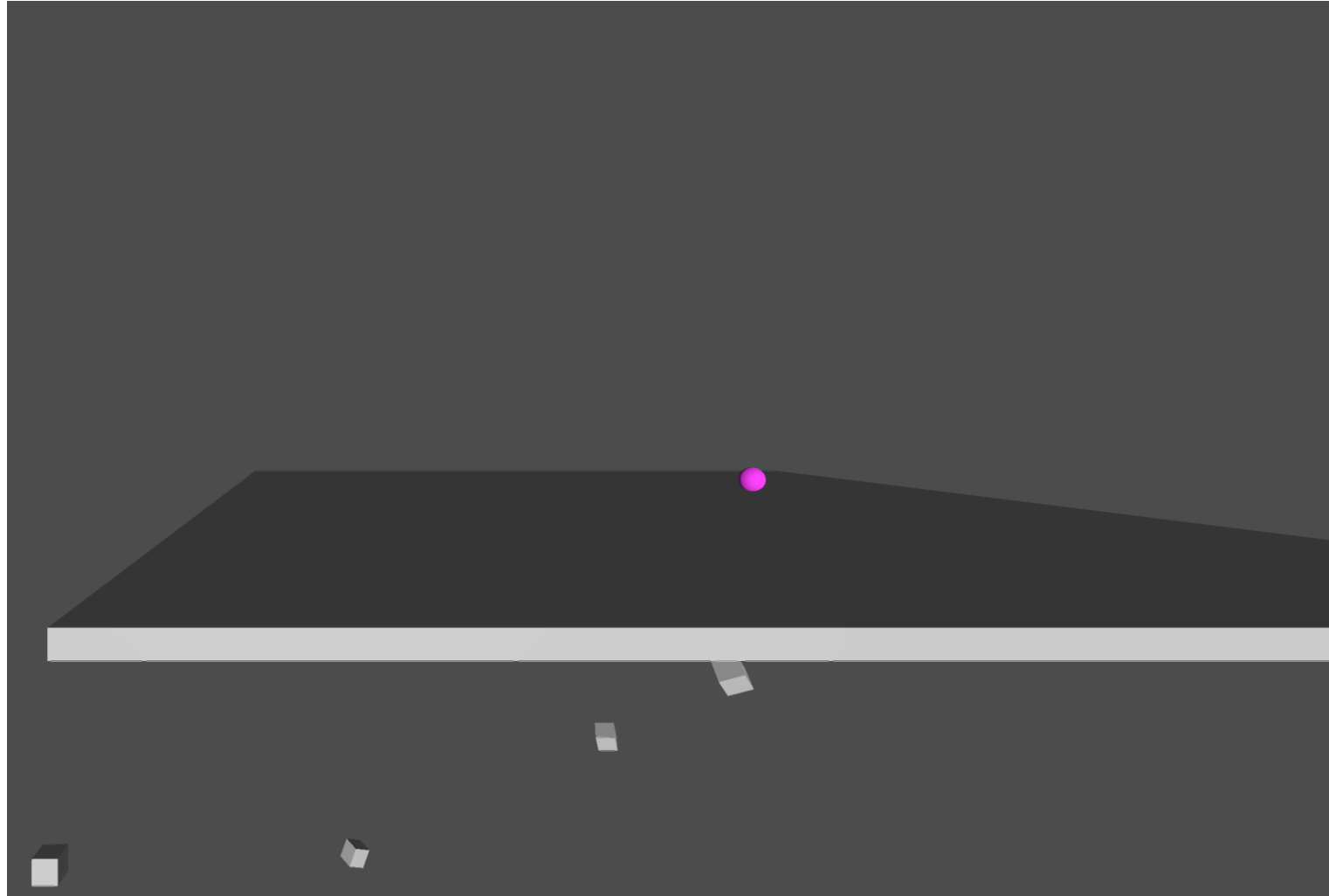
바닥과 부딪히지 않음: 바닥도 물리 객체로



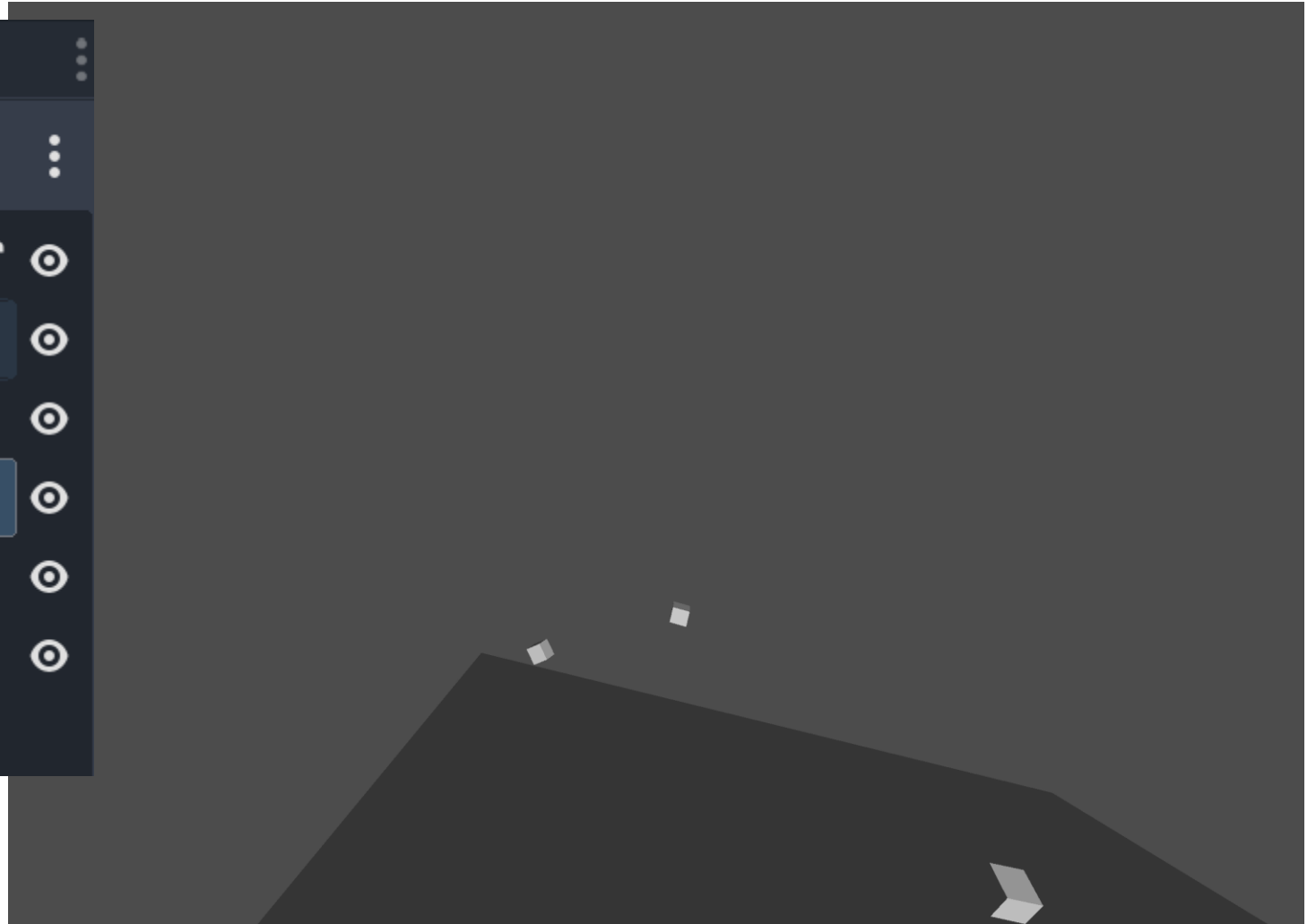
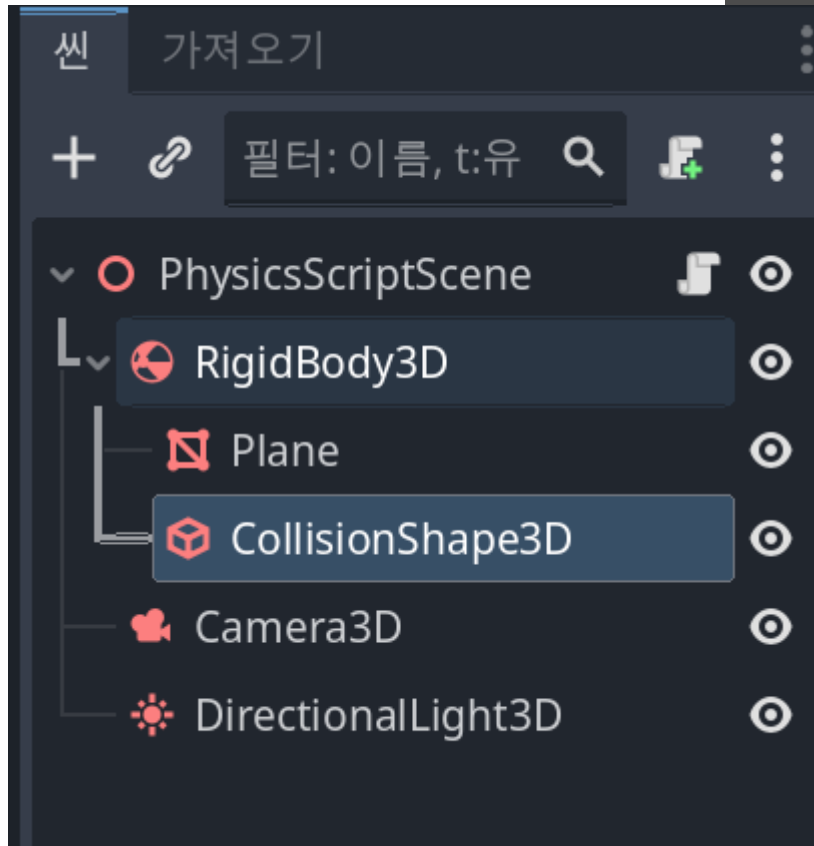
바닥도 Rigidbody로



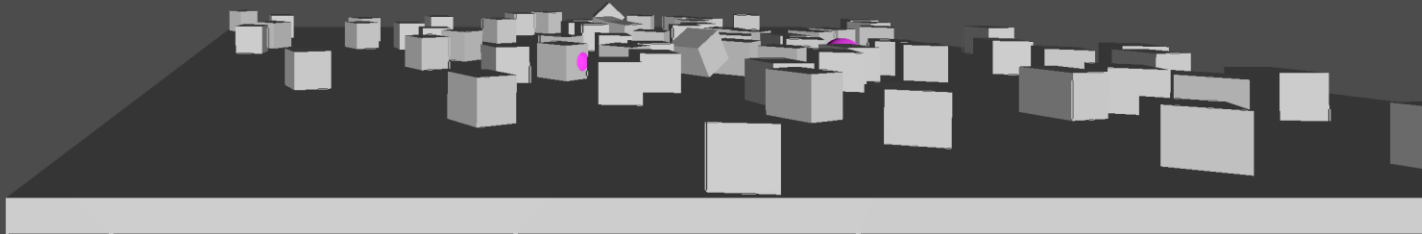
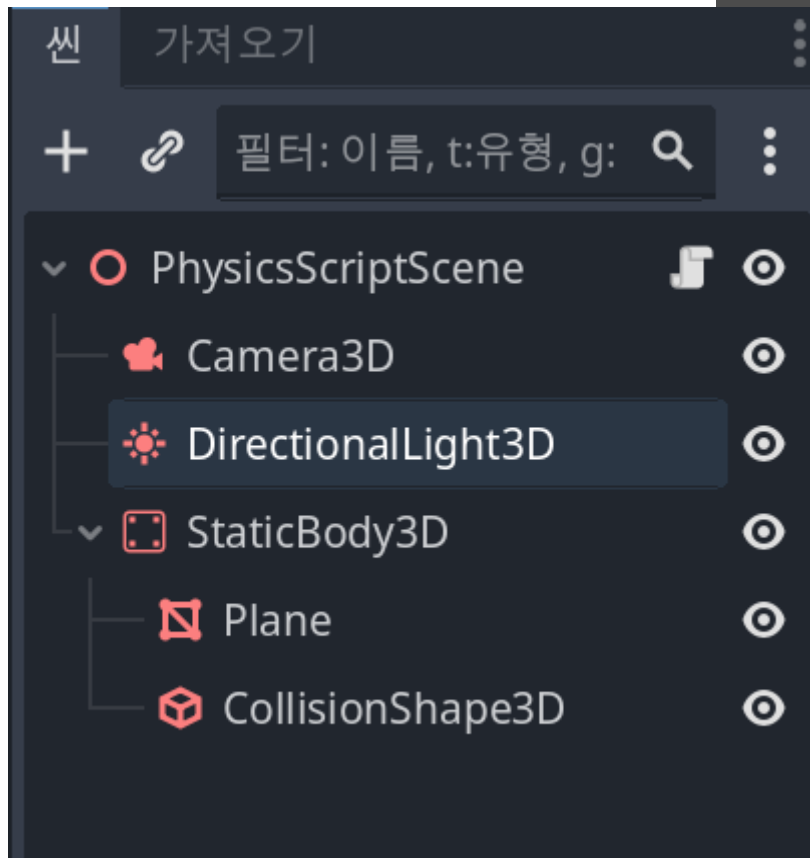
중력을 0으로 하면? 제자리에 있지만 충돌 반응 없음



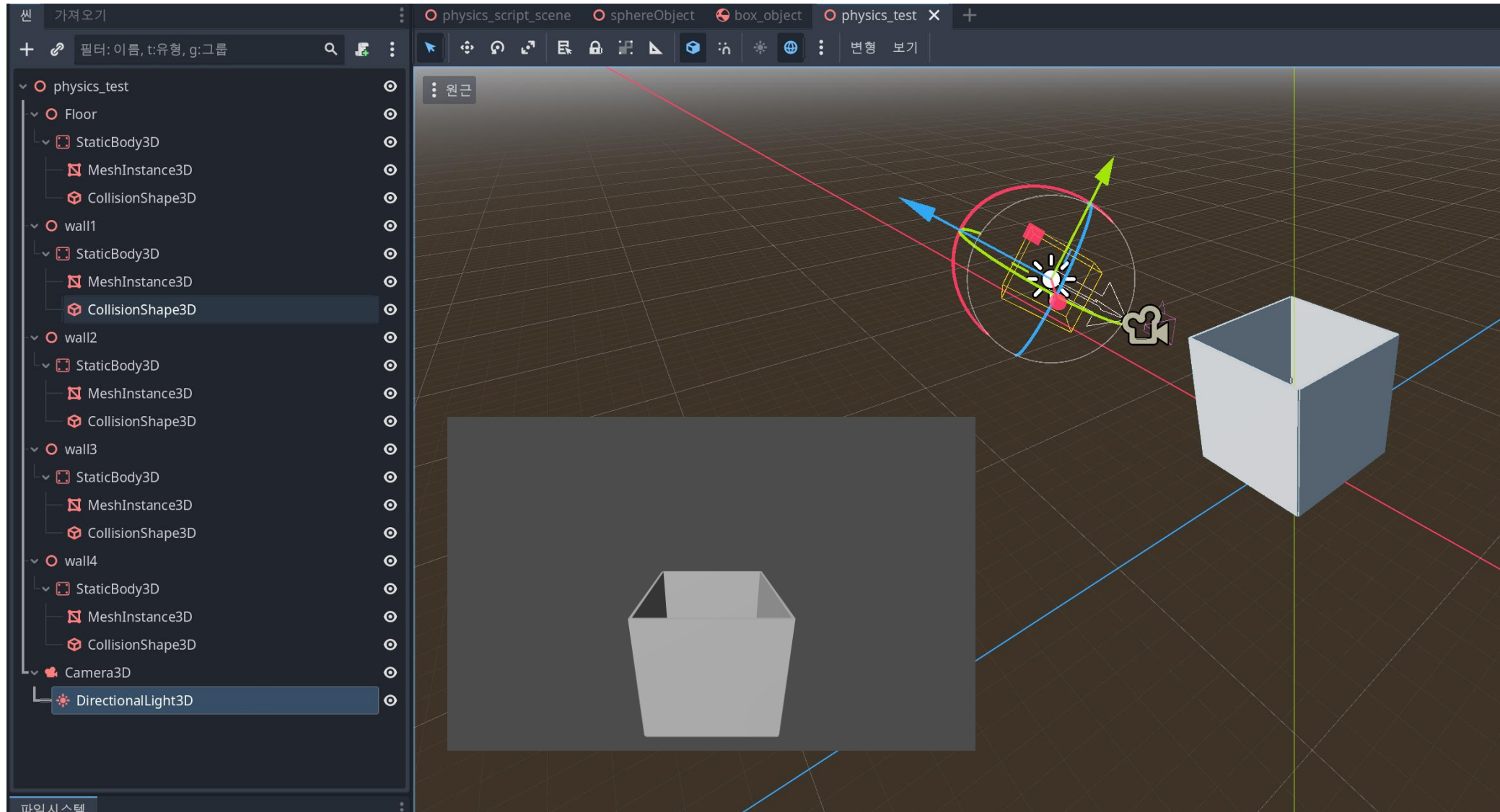
충돌 객체 추가 ➔ 충돌 시 무중력 상태에서 움직이듯 움직임



StaticBody3D

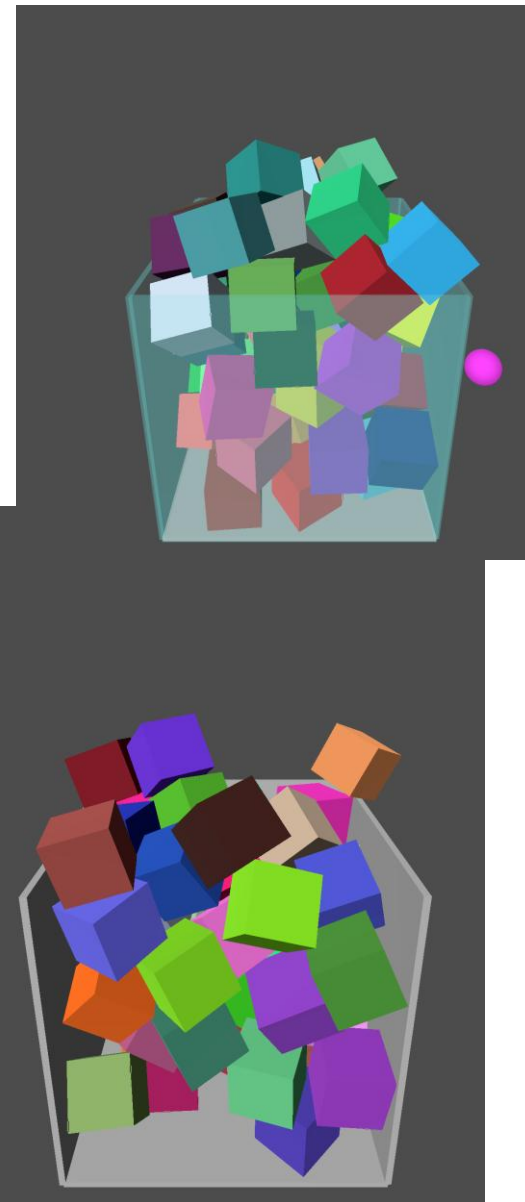


테스트 씬 생성

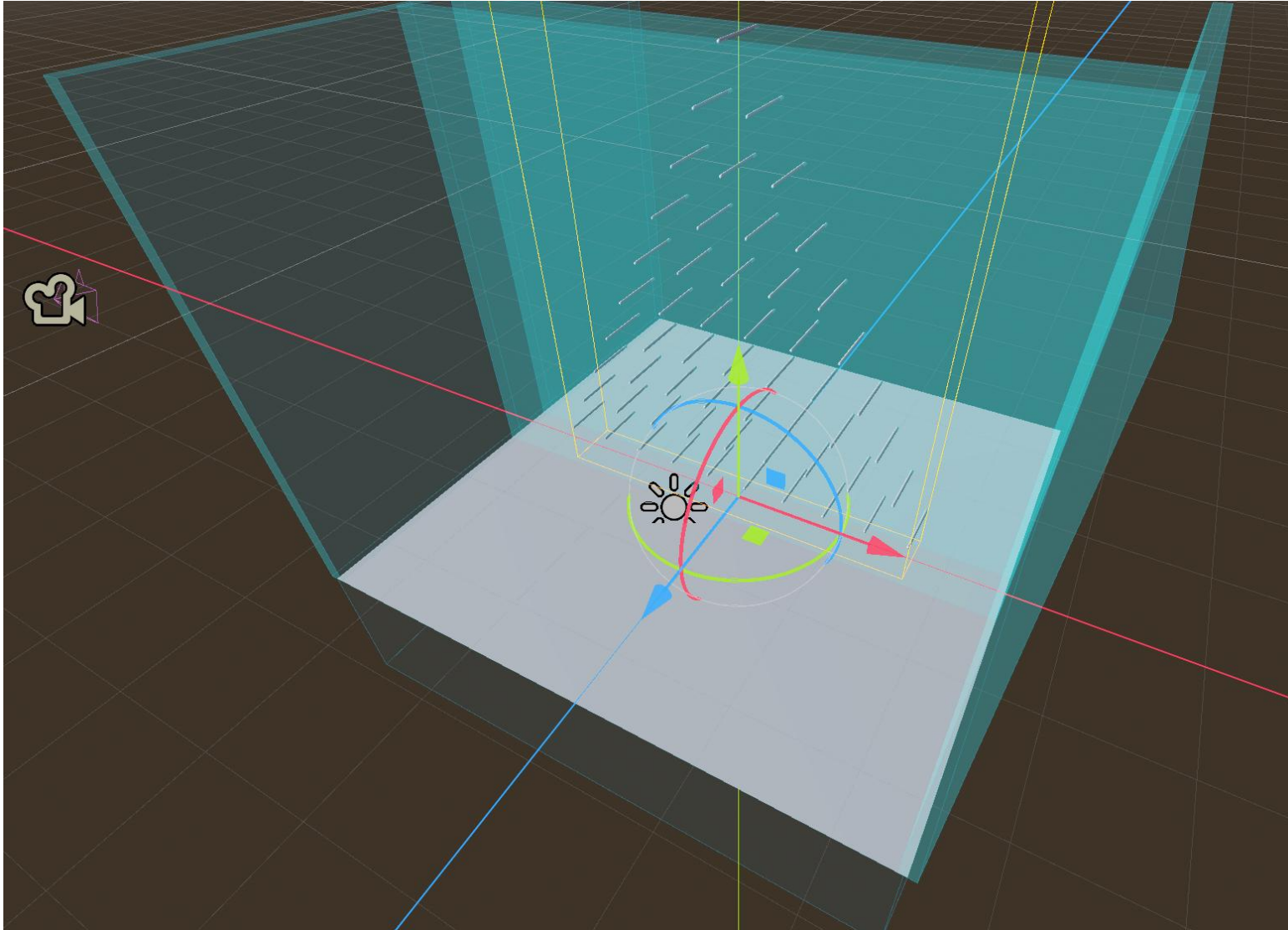


생성

```
func _spawn_box():  
    var obj = box_scene.instantiate() as RigidBody3D  
  
    obj.global_position = Vector3(0, 7, 0)  
  
    obj.linear_velocity = Vector3(randf_range(-2,2), 0, randf_range(-2,2))  
    obj.angular_velocity = Vector3(randf_range(-15,15), randf_range(-15,15), randf_range(-15,15))  
  
    # 색깔도 랜덤  
    if obj.has_node("MeshInstance3D"):  
        var mat = StandardMaterial3D.new()  
        mat.albedo_color = Color(randf(), randf(), randf())  
        obj.get_node("MeshInstance3D").set_surface_override_material(0, mat)  
  
    get_tree().current_scene.add_child(obj)
```

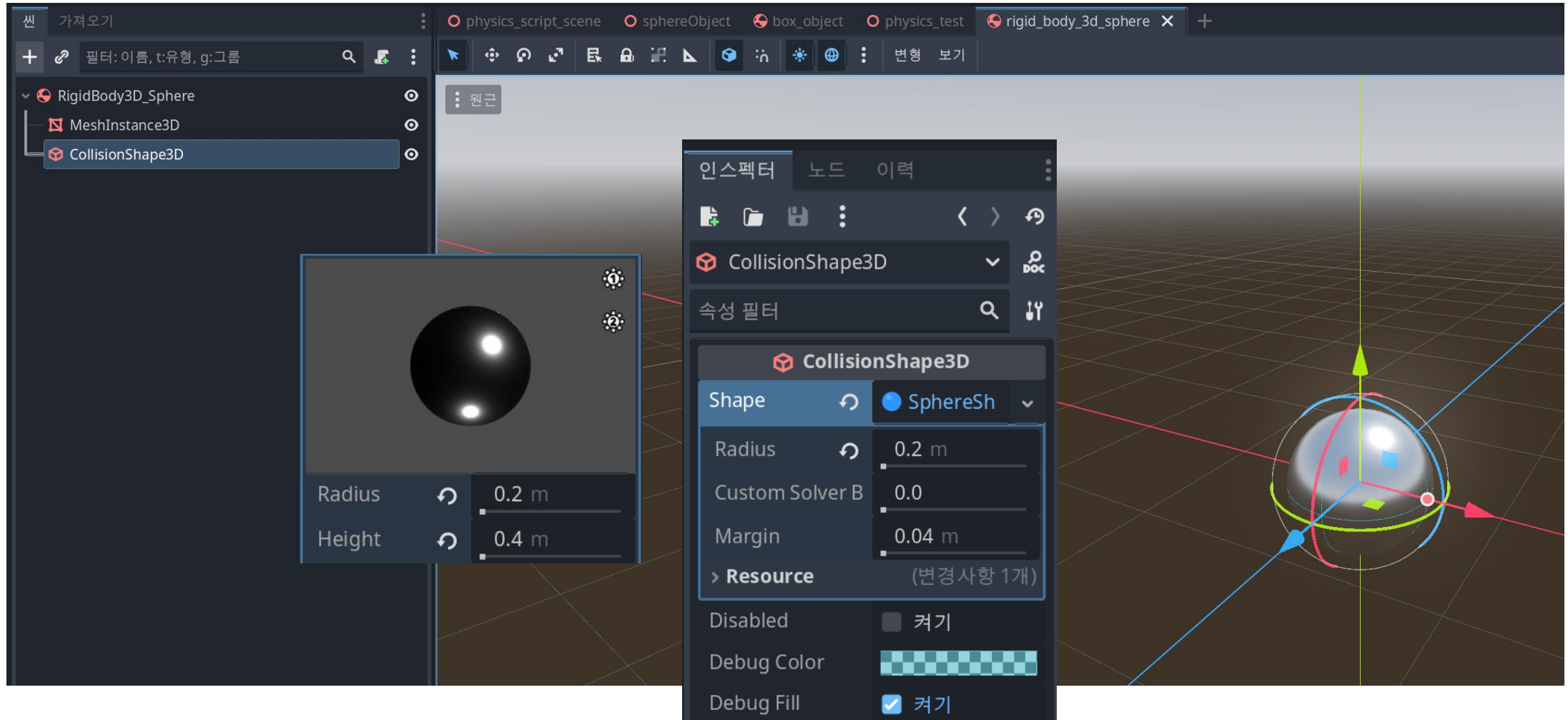


물리엔진 성능 테스트 – 구상하는 환경 (핀이 너무 많아...)



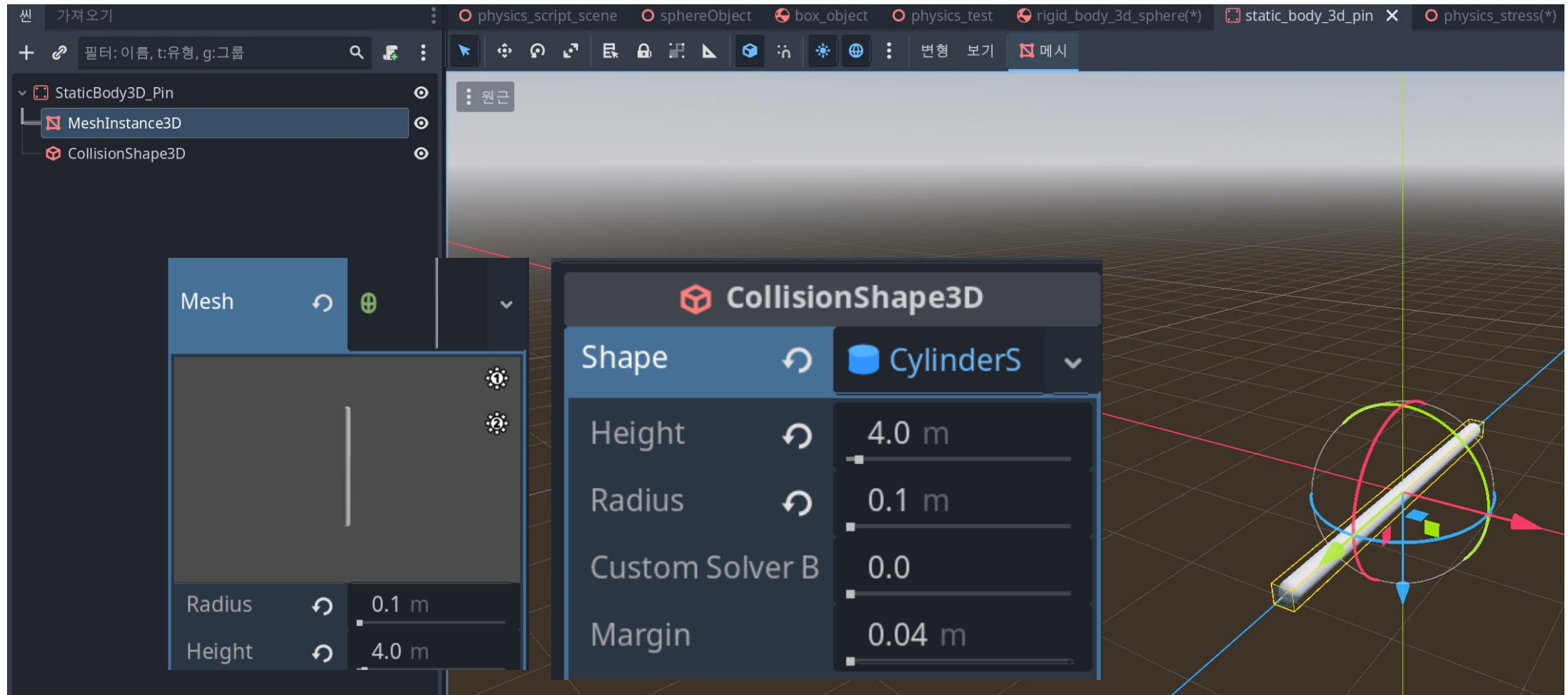
물리엔진 성능 테스트 –

강체 구를 만들자: rigid_body_3d_sphere.tscn

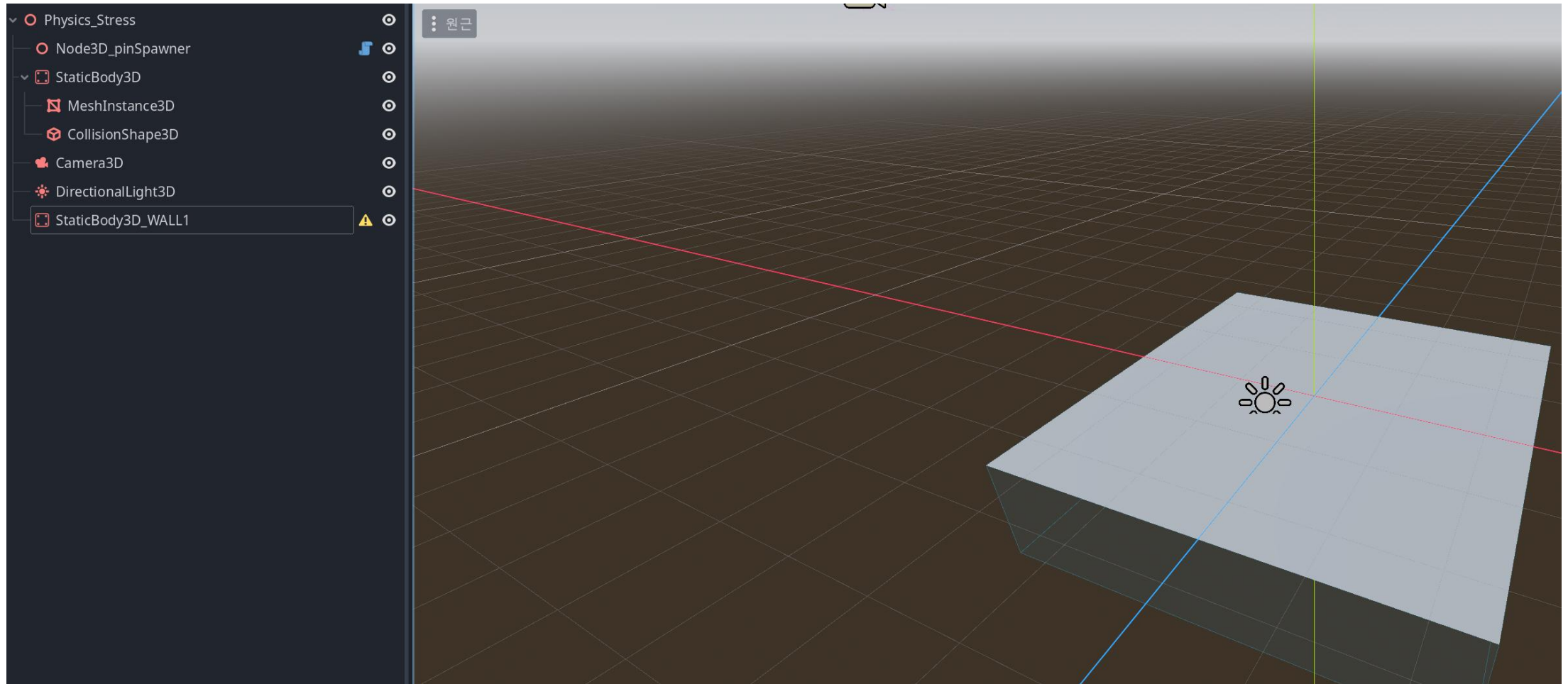


물리엔진 성능 테스트 –

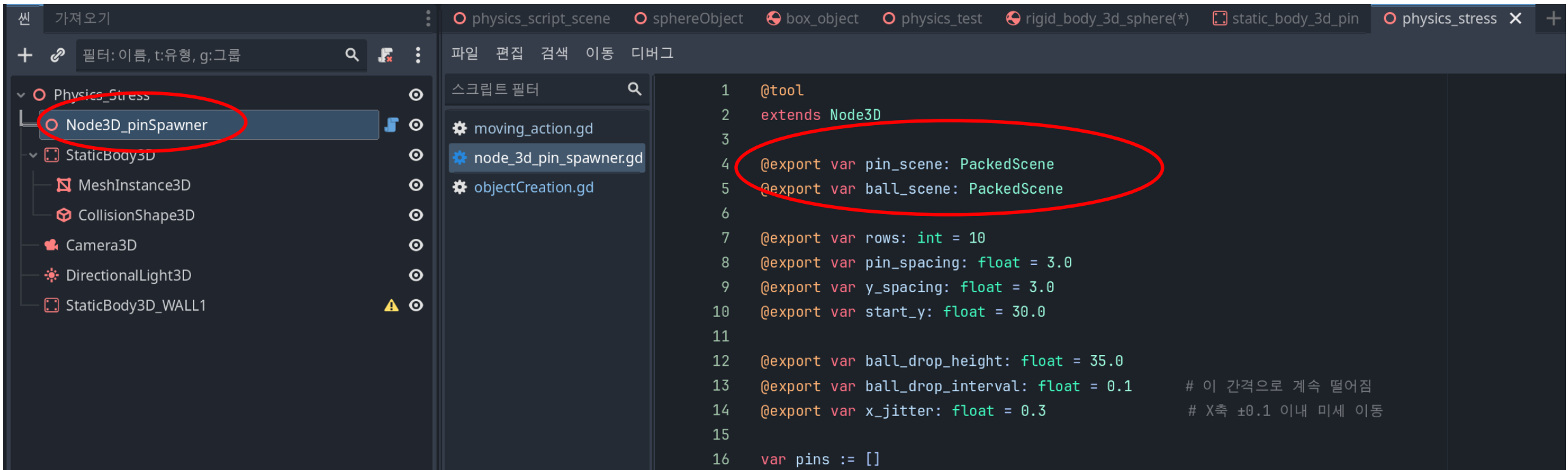
핀을 만든다: static_body_3d_pin.tscn



테스트할 장면을 만든다



테스트할 장면을 만든다



```
@tool  
extends Node3D
```

```
@export var pin_scene: PackedScene  
@export var ball_scene: PackedScene
```

```
@export var rows: int = 10  
@export var pin_spacing: float = 3.0  
@export var y_spacing: float = 3.0  
@export var start_y: float = 30.0
```

```
@export var ball_drop_height: float = 35.0  
@export var ball_drop_interval: float = 0.1    # 이 간격으로 계속 떨어짐  
@export var x_jitter: float = 0.3              # X축  $\pm 0.3$  이내 미세 이동
```

```
var pins := []  
var timer: Timer
```

```
func _ready() -> void:
```

```
    spawn_pins()
```

```
    start_ball_timer()
```

```
func start_ball_timer() -> void:
```

```
    # 기존 타이머 있으면 제거
```

```
    if timer and is_instance_valid(timer):
```

```
        timer.queue_free()
```

```
    timer = Timer.new()
```

```
    timer.wait_time = ball_drop_interval
```

```
    timer.one_shot = false
```

```
    timer.timeout.connect(_on_timer_timeout)
```

```
    add_child(timer)
```

```
    timer.start()          # ← 이게 핵심! 반드시 start() 호출
```

```
    print("공이 ", ball_drop_interval, "초마다 떨어지기 시작!")
```

```
func _on_timer_timeout() -> void:
```

```
    drop_one_ball()
```

```
func spawn_pins() -> void:
    clear_pins()
    for row in range(rows):
        var pins_in_row := row + 1
        var y := start_y - row * y_spacing
        var half := (pins_in_row - 1) * pin_spacing * 0.5

        for i in range(pins_in_row):
            var x := -half + i * pin_spacing
            var pin := pin_scene.instantiate() as Node3D
            add_child(pin)
            pin.position = Vector3(x, y, 0)
            pins.append(pin)

    print("핀 배치 완료: ", pins.size(), "개")
```

```
func drop_one_ball() -> void:
    if not ball_scene:
        push_error("ball_scene이 연결되지 않았습니다!")
        return

    var ball := ball_scene.instantiate() as RigidBody3D
    add_child(ball)

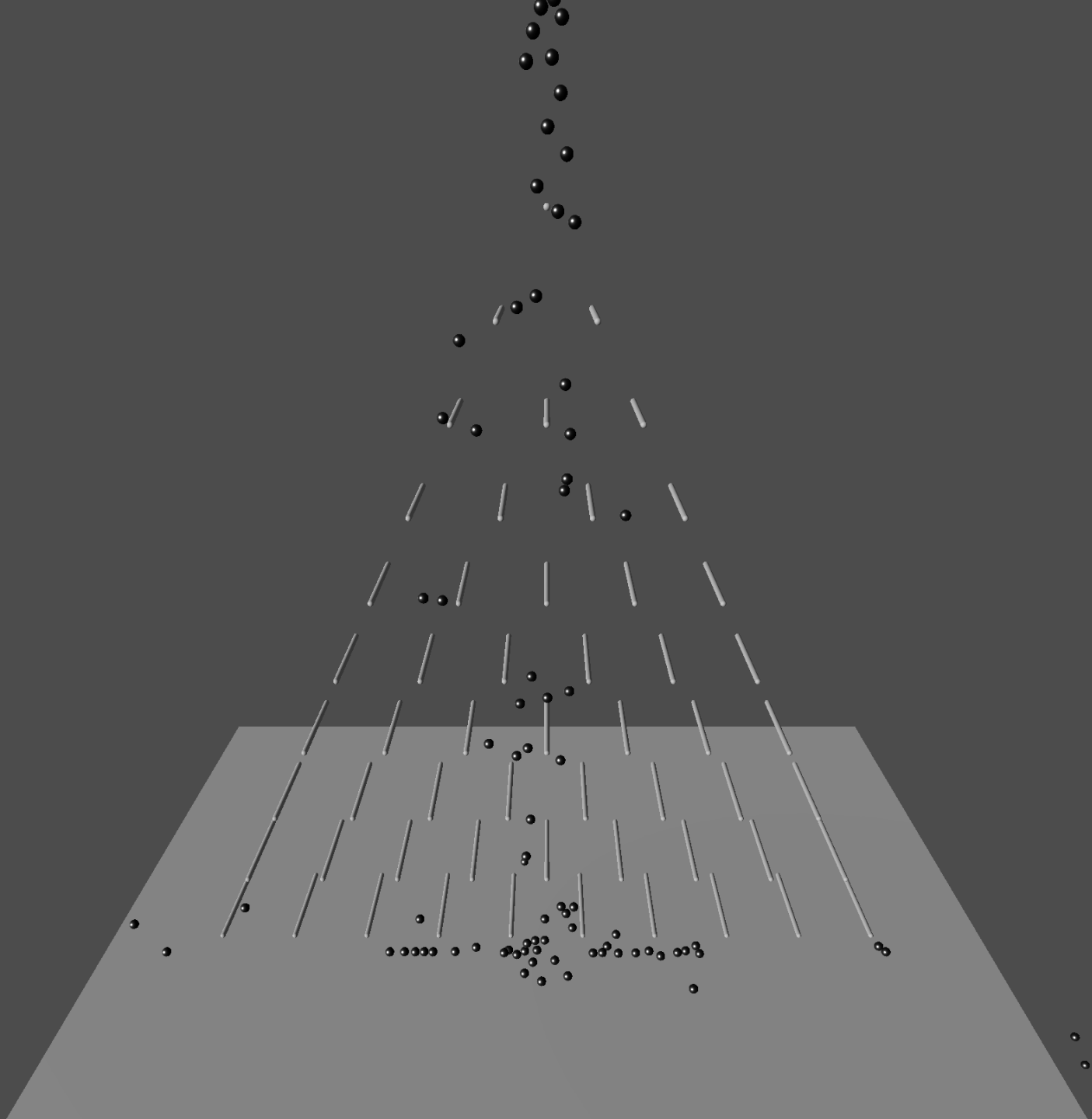
    # 정확히 (0, 35, 0)에서  $\pm 0.1$  이내 미세 랜덤
    var jitter_x := randf_range(-x_jitter, x_jitter)
    ball.position = Vector3(jitter_x, ball_drop_height, 0)

    # 거의 수직으로 떨어지게 (필요하면 완전 0으로 해도 됨)
    ball.linear_velocity = Vector3(randf_range(-0.3, 0.3), 0, 0)
```

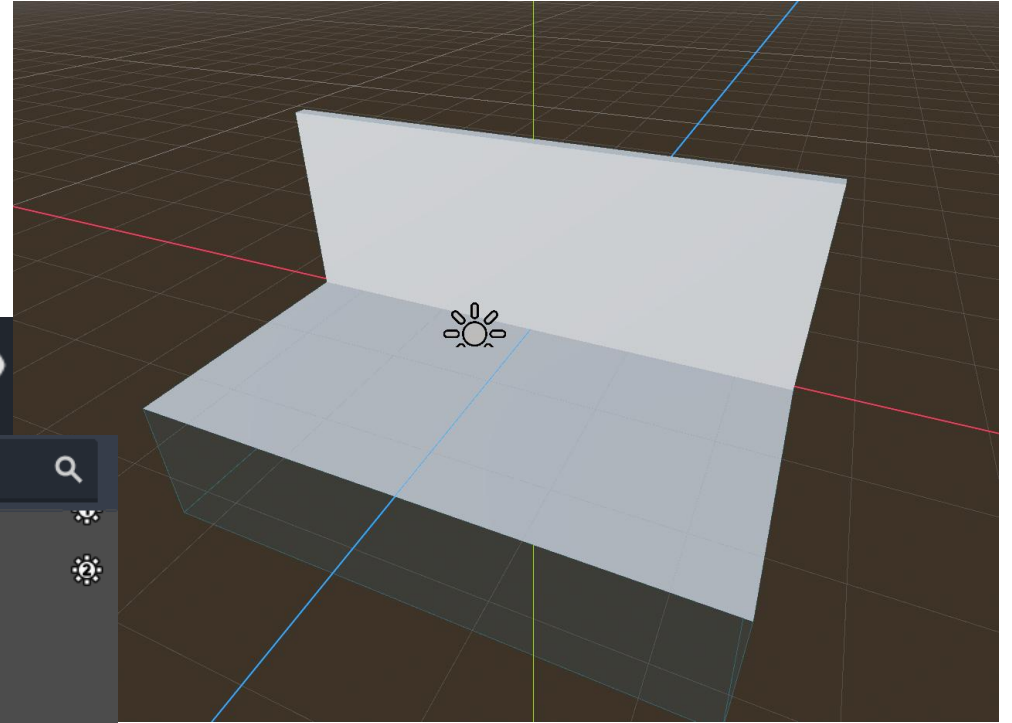
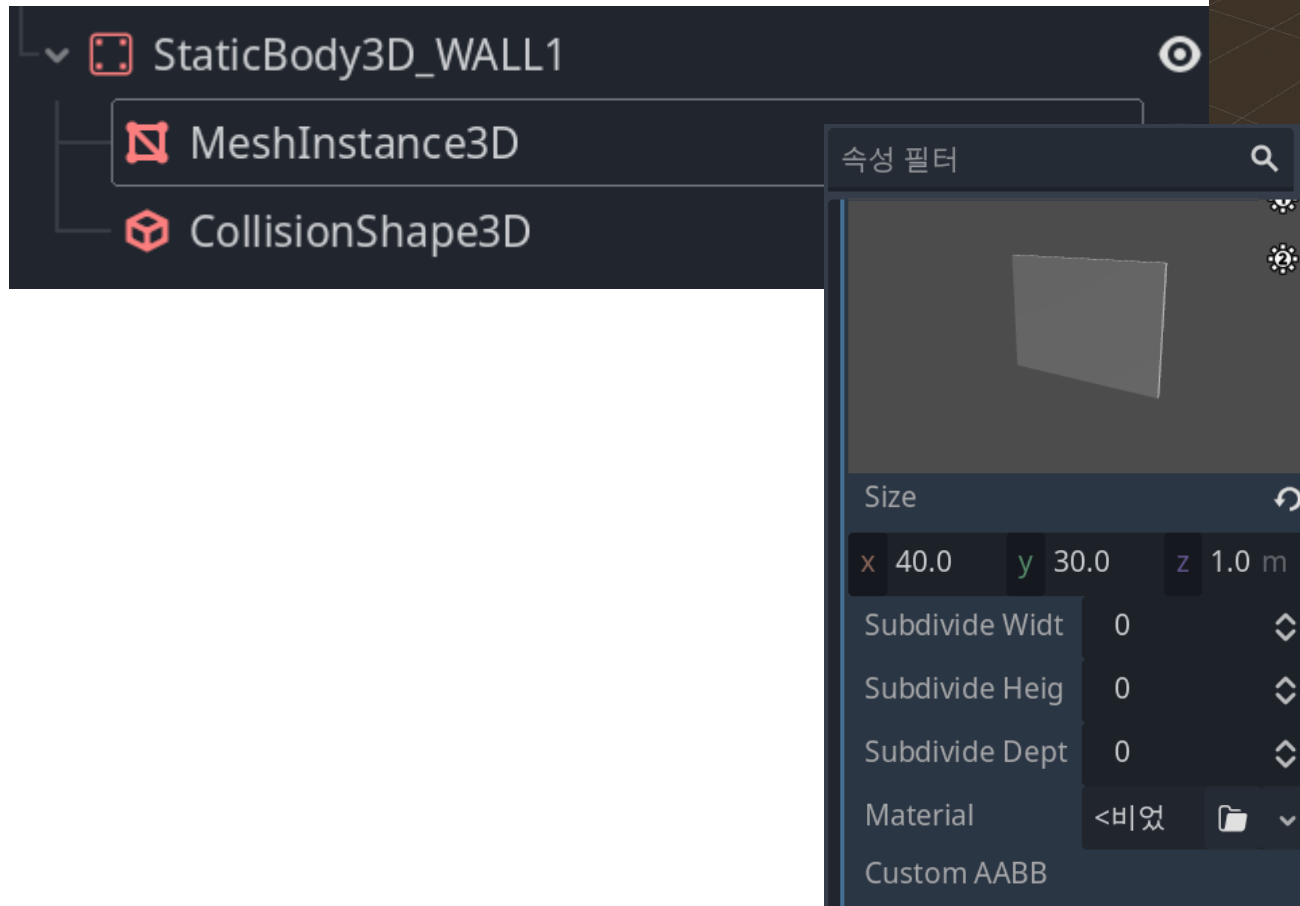
```
func spawn_pins() -> void:
    clear_pins()
    for row in range(rows):
        var pins_in_row := row + 1
        var y := start_y - row * y_spacing
        var half := (pins_in_row - 1) * pin_spacing * 0.5

        for i in range(pins_in_row):
            var x := -half + i * pin_spacing
            var pin := pin_scene.instantiate() as Node3D
            add_child(pin)
            pin.position = Vector3(x, y, 0)
            pins.append(pin)

    print("핀 배치 완료: ", pins.size(), "개")
```



벽을 만들어 가두어 보자



이런 장면을 원하지만,
속도가 느려지고 이상한 결과가 나올 수 있다: 물리엔진 교체

